Agent Self-Evolution

Phase 5 Validation Report

Continuous improvement loop via a propose-only triage sentinel

---

Date: June 25, 2026

Repository: github.com/jramos/agent-self-evolution

Executive Summary

Phase 4 proved the repair loop can fix a real broken tool when it is handed the failing test. Phase 5 ships the **supply side**: a propose-only triage sentinel that scans a target repository's recent git stream for bugs the validated loop *could* fix, ranks them, and writes a queue. It is the front-end that closes the continuous-improvement loop — detection feeding repair — while keeping every consequential decision in human hands.

The sentinel is shipped and operational. The scan is **free** (pure git, no LLM) and safe to schedule; the attempt step reuses the Phase 4 worktree → repair → oracle-gate pipeline **verbatim**, is **cost-capped**, and is triggered only by a human. It **never edits the target repository, never opens a pull request, and never deploys** — it surfaces candidates and the gate's verdict; a person decides what, if anything, to ship. The honest binding constraint is supply, not mechanism: reproducible bugs are scarce in a single repository's history (see Caveats), so the sentinel's job is to surface and safely gate the rare real candidate.

Key Result	Value
Triage sentinel	shipped — propose-only front-end to the repair loop
Scan cost	$0 (pure git, no LLM) — safe to schedule
Attempt step	human-triggered, cost-capped, never auto-PR
Repair + gate reused	the Phase 4 loop, verbatim
Binding constraint	supply of reproducible bugs (see Caveats)

Background

The code-evolution loop has two halves. Phase 4 is the **consumer**: given a bug with a failing test, repair it and gate the fix. Phase 5 is the **producer**: find bugs in a repository's git stream that look like work the consumer could do. Splitting them this way is deliberate — self-improvement is gated at the deployment step by a human, not at detection, so the producer is allowed to run cheaply and often while the consumer (which spends, and whose output could ship) stays a deliberate, supervised action.

Approach: Scan, then Attempt

The sentinel is two steps, exactly one of which costs money. The scan ranks candidates and writes a queue; the attempt runs the validated repair loop on the top K. Ranking is git-only and intentionally cheap — dependency-regressions first, then most recent — so a high rank means "look at this first," not "this will repair."

Step	Command	Cost	Who triggers

Scan (rank → queue)	monitor --repo <repo>	$0	safe to schedule
Attempt (repair top K)	… --attempt-top K --max-cost-usd C	spends	manual, gated

Each attempted candidate is annotated with a verdict; the loop records whether it *could* produce an oracle-matching fix, never a deploy.

Verdict	Meaning
attempted (+ deploy_reachable)	loop ran; majority of seeds gave an oracle-matching fix
not_valid	parent doesn't cleanly fail what the fix passes
source_missing / too_large	skipped before repair (source gone, or too big to rewrite)
cost_ceiling	the cost cap was hit; remaining candidates not attempted

Operation

Parameter	Value
Input	a target repo's git stream (default: last 90 days)
Scan output	triage_queue.json (machine-readable) + triage_report.md (ranked)
Ranking	dependency_regression > bug_fix, then most recent
Attempt cap	--max-cost-usd required whenever --attempt-top > 0
Repair + gate	the Phase 4 loop (worktree → repair → oracle gate), unchanged

The scan can be scheduled (a $0 weekly refresh); the spend step never can. The wrapper and launchd template are written to enforce scan-only scheduling — a copy-paste that added `--attempt-top` would turn a free job into unsupervised spend, so the design refuses it.

Results

The mechanism is validated end-to-end. The scan is deterministic and free, producing a ranked, jq-aggregable queue plus a human-readable report with a ready-to-run attempt command. The attempt step reuses the Phase 4 repair loop and deploy gate without modification, so its repair quality is exactly the validated 0.60–0.74 deploy-reachable rate, and its verdict taxonomy behaves as designed — rejecting `not_valid` candidates whose parent doesn't cleanly fail, and accepting only oracle-matching fixes. The result is a working continuous-improvement front-end: detection is automated and cheap; repair is validated and gated; deployment stays human.

Safety and Guardrails

Phase 5's safety story is its defining feature: the producer is **propose-only**. Every guarantee below is structural, enforced by the CLI and the scheduling templates, not by convention.

Guarantee	Status
Never edits the target repository	enforced
Never opens a PR or deploys a fix	enforced
Scan never spends (pure git, no LLM)	enforced
Attempt refuses to run uncapped	enforced (--max-cost-usd required)
Scheduled jobs scan only	enforced (wrapper + template)

Caveats and Limitations

The mechanism works; the open question is how much real work exists for it. The supply of *reproducible* bugs — a buggy parent, an upstream-fix oracle, and a red→green test — is structurally scarce, and this bounds how far an autonomous loop can go.

Supply probe	Result
Hermes, one year of history	0 clean tool-local dependency regressions
Cross-repo port (encode/httpx)	0 / 35 valid organisms — KILL
dependency_regression label	noisy — fires on any manifest touch

Two things follow. First, the `dependency_regression` rank is a hint, not a guarantee: it fires when a fix commit also touched a dependency manifest, and a year-long analysis of one active repo found **zero** clean, tool-local dependency-version regressions — the manifest-touching commits were broad migrations or feature additions. Second, the clean oracle-bearing supply is a property of hermes-agent's tool-registry architecture (isolated single-file tools, name-matched tests), not a general feature of mature Python repos: a port to `encode/httpx` found abundant bug-fix-shaped commits but **0 of 35** reproduced as a single-tool red→green organism (pre-registered verdict: KILL). This is the **double wall** — independent verification and bug supply — that bounds autonomous continuous improvement. The sentinel's value is to surface and safely gate the rare real candidate, not to manufacture a stream of them; a portable supply would need SWE-bench-grade harvesting (issue→PR linkage + per-commit environments), which the project composes with rather than rebuilds.

Roadmap

Phase	Target	Engine	Timeline	Status

Phase 1	Skill files (SKILL.md)	DSPy + GEPA	3-4 weeks	Validated ✓
Phase 2	Tool descriptions	DSPy + GEPA	2-3 weeks	Validated ✓
Phase 3	System prompt sections	DSPy + GEPA	2-3 weeks	Validated ✓
Phase 4	Tool implementation code	Iterative test-feedback repair	3-4 weeks	Validated ✓
Phase 5	Continuous improvement	Propose-only triage sentinel	2 weeks	Shipped ✓

With Phase 5 the loop is end-to-end: detect (free, scheduled) → repair (validated, cost-capped) → gate (resists a lying green) → human deploy. The framework can now find its own work; how much work there is remains a supply question, stated plainly above.

Immediate Next Steps

- **Broaden the supply.** Adopt an external fail→pass feed (e.g. SWE-smith's per-repo environments) so the sentinel ranks a real backlog, instead of manufacturing one from a single repo's thin stream.
- **Worktree-based difficulty ranking.** Today's rank is git-only; a cheap worktree probe could estimate repair difficulty so the top of the queue predicts "will repair," not just "look first."
- **The untested artifact surface.** Novel MCP tool descriptions are the one instructions surface not yet measured; enrolling them would extend the sentinel beyond code.